

Phase 3 Post-Mortem

Tenant Isolation at the Data Layer

Date: 2026-09-14

Commit: 24cf950 on main

Scope: Internal architecture. Zero user-facing behaviour change for the current admin session. Real change surfaces when a second operator logs in.

The one-paragraph version

Before Phase 3, every operator role read the same global `EVENTS` array raw. If CPH Airport and Esbjerg Harbour were both onboarded to the same deployed instance, the code paths would happily show CPH's operator the Esbjerg events. Tenant separation existed only at render time (`visibility.js` redacted things after the data had already flowed through the UI). Now every event is stamped with an immutable `tenantId` at creation, and every read function inside `events.js` filters visibility against an ambient current actor. Cross-tenant events stop existing before they leave the module. When Azure auth lands, the boot code swaps four lines and the same filter runs against a JWT-derived actor.

Why this mattered

Two problems with the pre-Phase-3 arrangement:

Problem 1: render-time redaction is fragile.

The data flowed unfiltered from `events.js` to every render function. Any place that forgot to check `visibility.js` — or checked it incorrectly — leaked cross-tenant data. Auditing 22,000 lines of `main.js` for every possible leak path is not a security model. It's a security fingers-crossed.

Problem 2: pre-existing tenant-scoping helpers were unwired.

`main.js` already had `_tenantForSite` and `_tenantScopedLinkedIds` as private helpers, added defensively over time. They just weren't attached to the read seam. Phase 3 lifts these into `events.js` so they run automatically on every read.

Before vs After — one concrete example

Before — an operator role clicks on an event that xlink-chains into another operator's site:

```
// main.js: builds the chain view
const chain = eventsInSameChainAs(currentEventId);
// chain now contains events from tenants the operator can't see.
// Rendering code depends on visibility.js catching this at redact time.
// If visibility.js has a bug or if a render surface doesn't call it,
// cross-tenant data leaks into the UI.
```

After — same call:

```
// events.js under the hood applies _visibleToActor filter.
// The chain query returns only events the current operator's tenant
// can see. Cross-tenant links are simply not in the result set.
// No render code has to remember to check anything.
```

The API stayed identical. The invariant moved from "hopefully every render surface checks visibility" to "the data layer guarantees you never see it in the first place."

The three-tenant model

Kind	Sees what	Filter mechanism
Admin (ISR itself)	Everything, across every tenant	<code>isAdminBypass</code> — skip filter
Operator (CPH, Esbjerg, Energinet, Billund)	Only events in their own tenant	<code>event.tenantId === actor.tenantId</code>
Receiver (Politi, BRS, PET, FE, ...)	Only events cascaded to their destination	<code>event.escalations[].destinationId ∈ actor.destinationIds</code>

Receiver-side isolation already worked pre-Phase-3 via `eventsForDestinations`. Preserved verbatim. Operator-side isolation is the new work.

The mechanism

Ambient actor context

`events.js` holds a module-level `_currentActor` variable. `main.js` sets it once at boot from `getActiveRole()` and re-sets it every time the role dropdown changes. Every read function in `events.js` applies the filter against `_currentActor` automatically.

This is the same pattern server-side auth uses: JWT verification middleware sets `req.user`, downstream handlers read it implicitly. When Azure auth lands, the boot code becomes: read the session JWT, build the actor from claims, call `setCurrentActor`. Every event-reading function in the platform continues to work without a single signature change.

Actor derivation

```
role from dropdown → actorFromRole(role) → {kind, roleId, tenantId, siteIds/destinationIds}
```

For operators, `tenantId` is derived from the operator's site manifest: `SITES[role.siteIds[0]].operatorAccountId`. All sites belonging to one operator share the same tenant id — enforced by the safety net check below.

Tenant stamping

Every event created via `addEvent` gets `event.tenantId = tenantForSite(event.siteId)` stamped at creation. Immutable once written. Seed events (the demo data hardcoded at the top of `events.js`) get backfilled at module load, so even the demo events have the correct tenant scope.

The 4 read functions that changed

Function	Old behaviour	New behaviour
<code>getEvent(id)</code>	Raw lookup	Filters via <code>_visibleToActor</code>
<code>filteredEvents()</code>	Filters by classification	Also filters by tenant
<code>getSelectedEvent()</code>	Raw lookup	Routes through <code>getEvent</code>
<code>unionLinkedEventDomains(id)</code>	Unions all linked	Operator only unions same-tenant links

`eventsForDestinations()` stayed as-is — its destination filter already IS the receiver tenant scope.

The safety net

Two build-time checks now run before Vite ever fires:

```
$ npm run build
```

```
✓ Event mutation policy satisfied. Every event write goes through events.js. (Phase 1)
✓ Tenant-stamp policy satisfied. 9 manifests, 4 operators, 4 distinct tenants – all aligned. (Phase 3)
```

The Phase 3 check verifies:

1. Every site manifest declares a `tenant:` field.
2. Every operator id in `roles.js` matches at least one manifest tenant.
3. Every manifest tenant matches at least one operator id.

Any mismatch fails the build with a pointer to the file and the specific misalignment.

Two typos were caught during Phase 3 landing:

- `sites/cph.yaml` said `op-cph-airport` but `roles.js` said `op-cph-airports` (plural). Fixed the manifest.
- `sites/esbjerg.yaml` said `op-esbjerg-havn` but `roles.js` said `op-esbjerg-port`. Fixed the manifest.

Without the safety net these would have silently broken tenant filtering — CPH operator would have logged in and seen an empty inbox forever, with no error anywhere.

What this unlocks (immediately)

For Azure auth

The Azure JWT verification middleware becomes a four-line swap in `main.js`:

```
// Before:
setCurrentActor(actorFromRole(getActiveRole()));

// After:
const claims = await verifyAzureSession();
setCurrentActor(actorFromClaims(claims));
```

Every read function keeps working. Every write function keeps working. The only work is adding `actorFromClaims` next to `actorFromRole`. That's it.

For onboarding a second operator

Today you could technically deploy this platform for two operators (CPH and Esbjerg) in one instance and it would work correctly. Before Phase 3, this required manual auditing of every render surface for leaks. Now it's a matter of adding both operators to `roles.js`, ensuring their manifests point at the correct tenant, and letting the safety net verify.

For xlink cross-tenant chains

An operator can no longer see the full chain when a drone crosses into another tenant. This is the correct behaviour for critical infrastructure — CPH doesn't need to see the Energinet event details. But both parties still get the coordination they need: they each cascade to a shared receiver (PET, FE, Rigspoliti), and the shared receiver sees BOTH events via receiver-side isolation. Cross-tenant coordination happens through destinations, not through direct data sharing.

What this does NOT do

- **No server-side enforcement.** This is still all client-side. A determined attacker can open the console and call `setCurrentActor({kind: 'admin', isAdminBypass: true})` — no auth. When Azure lands, the JWT

verification server-side becomes the primary gate; the client-side filter becomes defence-in-depth.

- **No user management or auth UI.** Roles are still hardcoded in `roles.js`. No invite flow, no SSO.
- **No event-level compartments.** Every event in a tenant is visible to every role within that tenant. Fine-grained per-event permissions (e.g. "this event is compartmented to PET only") are handled by the existing `visibility.js` chapter-level system and are unchanged.
- **No nested-field tenant enforcement.** If a UI directly accesses `event.someField` after fetching a legitimate event, all of that event's fields are visible. Field-level redaction stays where it always was: `visibility.js` at render time.

The verdict

Metric	Before	After
Operator can read cross-tenant events at data layer	Yes (relied on render-time redaction)	No
Read functions with tenant filter	0	4
Xlink chain crosses tenant boundary in operator's view	Yes	No (only same-tenant links surface)
Site manifest → operator role alignment check	None	Build-time safety net
Boot-time actor context set	No	Yes (from <code>getActiveRole()</code>)
Role dropdown re-scopes actor	No	Yes (via <code>onRoleChange</code>)
Azure auth swap effort	Rewrite every read surface	Swap four lines in <code>main.js</code>
Manifest naming drift caught	None	2 typos fixed (CPH + Esbjerg)
User-facing behaviour under admin	Baseline	Identical
Lines added	—	253
Lines removed	—	7

Phase 3 done. Phase 2 (carving rendering out of `main.js`) is the last of the three internal refactors. It's the longest and highest-risk of the three, but Phase 1 and Phase 3 both landed under budget, and Phase 2 doesn't block any external work — it's an iteration-speed investment.

The bigger picture: with Phase 1 (write seam) and Phase 3 (read filter) both landed, the platform is now ready for Azure to attach a real auth layer + server-side persistence without touching a single line of application code. The seams are all in place.